GS — CLOUD SECURITY PREVENTION

e DEVSECOPS

RELATÓRIO TÉCNICO

Bruno Kenji Sakimoto

RM567502

1. Resumo executivo

Este projeto implementa um planner de tarefas web com interface React e uma API REST para persistência em SQLite. A implantação foi preparada para concentrar front-end e API em um único Azure App Service Linux, executado por um processo Node.js. O GitHub Actions automatiza build, testes, empacotamento e publicação.

A aplicação usa Azure App Service Linux, runtime Node.js 24 e região East US 2. Inicialmente foi usado o plano Free (F1); para testar a aplicação, posteriormente foi utilizado o plano B1 (Basic, pago).

Objetivo e escopo

Demonstrar uma aplicação web com separação entre interface e API, persistência de dados, integração contínua e hospedagem como serviço gerenciado. O relatório descreve os componentes e as evidências da implantação concluída.

Repositório

https://github.com/Brun0Kenji/GS-CloudSec

2. Arquitetura da aplicação

O navegador acessa a aplicação por HTTPS. O processo principal Node.js inicia app-host.js, que encaminha as rotas /api para o Express, publica os arquivos gerados do front-end e envia as demais rotas ao middleware Nitro/TanStack Start. Isso concentra front-end e API em um único App Service.

Componente	Responsabilidade
Navegador	Carrega a interface e chama endpoints relativos em /api.
Azure App Service	Hospeda o processo Node.js e fornece a URL HTTPS gerenciada.
app-host.js / Express	Ponto de entrada único; monta a API, serve .output/public e encaminha as outras rotas ao front-end.
API Express	Implementa operações REST de tarefas e subtarefas sob /api.
SQLite	Armazena registros em um arquivo tarefas.db no caminho definido por DB_PATH.

Fluxo principal: navegador -> HTTPS -> App Service -> (front-end TanStack Start ou API Express em /api) -> arquivo SQLite. O front-end e a API compartilham o mesmo domínio, então o build de produção usa VITE_API_URL=/api e não depende de CORS entre serviços.

Rotas da API

A API inclui status em GET /api/, listagem em GET /api/tarefas, consulta por identificador, criação (POST), atualização (PUT) e exclusão (DELETE). As operações usam respostas HTTP e JSON; a criação requer um título.

3. Linguagens e tecnologias

Tecnologia	Uso neste projeto
TypeScript	Linguagem predominante do front-end e das rotas da aplicação.

React	Biblioteca de componentes da interface.
TanStack Start / Router	Renderização e roteamento da aplicação; build gerado com Vite/Nitro.
JavaScript / Node.js 24	Runtime e servidor de aplicação; ponto de entrada app-host.js.
Express	Servidor HTTP e roteamento da API REST.
SQLite / SQL	Banco relacional embarcado em arquivo, acessado por node:sqlite.
GitHub Actions	Automação de instalação, build, testes e publicação no Azure.

O runtime de hospedagem configurado é Node.js 24. O código da API usa JavaScript e o módulo nativo node:sqlite; não há um servidor de banco de dados separado nessa arquitetura.

4. Nuvem, plano e armazenamento

Azure App Service (PaaS)

O Azure App Service é uma plataforma como serviço (PaaS): o Azure administra a infraestrutura física e parte da camada de sistema operacional/runtime, enquanto a equipe configura o aplicativo, o runtime, variáveis de ambiente e implantação. Não é uma máquina virtual criada ou administrada pelo projeto.

O plano atual é B1, na camada Basic. B1 é o SKU do plano do App Service, não de uma máquina virtual. O plano Free (F1) aparece apenas em capturas históricas anteriores à mudança.

Cotas observadas no portal

A Figura 1 registra as cotas do plano Free (F1) antes da mudança para B1. Portanto, os valores da captura são históricos e não descrevem o plano atual. O B1 foi escolhido para testar a aplicação com o tier Basic.

Persistência de dados

SQLite grava em arquivo. O código usa DB_PATH e, sem configuração, aponta para ./tarefas.db; no App Service isso pode ficar no diretório de publicação e não deve ser tratado como armazenamento permanente. Configure DB_PATH=/home/tarefas.db e confirme que o armazenamento persistente do App Service está habilitado para /home.

Não foi identificado um Azure Storage Account ou Azure SQL Database como parte da solução atual. O arquivo SQLite é adequado para uma demonstração de instância única, mas não é um banco compartilhado entre múltiplas instâncias. Para alta disponibilidade, escala horizontal, cópias de segurança ou uso concorrente maior, recomenda-se migrar para uma base gerenciada, por exemplo Azure SQL ou Azure Database for PostgreSQL, e planejar backups.

5. Build e implantação contínua

O workflow único .github/workflows/deploy-app.yml executa o processo abaixo na branch main ou por acionamento manual:

- Obtém o código e configura Node.js 24 no runner do GitHub Actions.
- Instala dependências reproduzíveis com npm ci.
- Gera o front-end com VITE_API_URL=/api e executa npm test.
- Remove dependências de desenvolvimento, prepara o pacote de produção com app-host.js, API, node_modules e .output.

- Publica o pacote no App Service pelo azure/webapps-deploy@v3.

Configuração do repositório: a variável de Actions APP_NAME deve conter o nome curto exato do App Service; o secret APP_PUBLISH_PROFILE deve guardar o perfil de publicação XML correspondente. O arquivo XML é uma credencial: mantenha-o apenas no secret do GitHub e nunca o inclua no repositório ou neste relatório.

No App Service, o comando de inicialização esperado é npm start. O script start executa node app-host.js. Configure também DB_PATH=/home/tarefas.db e valide a persistência de /home.

Segurança e operação

- Não compartilhar publish profiles, senhas, tokens ou identificadores de assinatura em capturas públicas.
- Aplicar privilégio mínimo aos acessos do repositório e rotacionar o perfil de publicação se ele for exposto.
- Confirmar HTTPS, variáveis de ambiente e logs de inicialização após cada deploy.
- Usar plano com recursos e disponibilidade compatíveis com o objetivo antes de tratar a aplicação como serviço de produção.

6. Estado final da aplicação

A execução final do GitHub Actions foi concluída, e a aplicação funcionou corretamente após o deploy. A captura final e o vídeo em anexo confirma que a interface carregou com o estilo visual esperado.

Validação final registrada

- Deploy da versão integrada realizado pelo workflow do GitHub Actions.
- Interface acessada pela URL HTTPS do App Service e apresentada com o layout estilizado.
- Arquivos CSS e JavaScript compilados servidos pelo host Express após a correção implementada.
- Os testes locais anteriores confirmaram CSS com status 200 e MIME text/css, além de respostas das rotas da API.

Conclusão: a arquitetura de instância única foi implantada e o pipeline automatiza o deploy. A captura final confirma que a interface está estilizada, e o responsável confirmou que a aplicação funcionou corretamente. O plano B1 (Basic, pago) foi escolhido para testar a aplicação. O SQLite requer armazenamento persistente e foi mantido em uma instância; essas limitações devem ser consideradas caso a solução evolua para produção.

7. Registros visuais

As imagens registram etapas da implantação: cotas do plano F1, execução bem-sucedida do workflow e estado final estilizado da aplicação, após a resolução do problema de CSS.

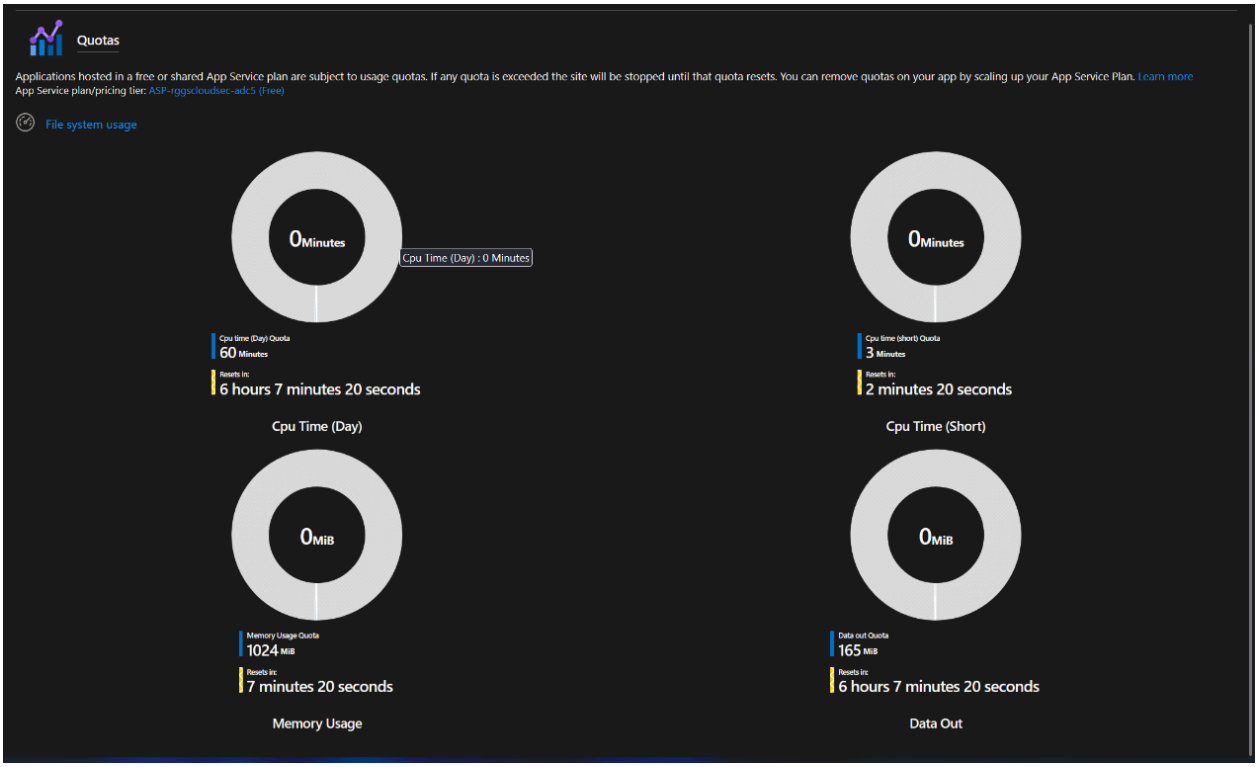


Figura 1 - Cotas do plano Free (F1) registradas antes da mudança para B1; captura histórica, não representa a configuração atual.

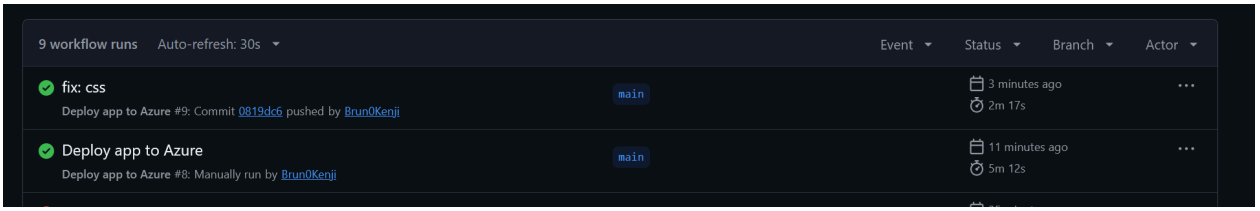


Figura 2 - Execução do workflow registrada como concluída no GitHub Actions.



Figura 3 - Estado final do planner no App Service HTTPS, com estilos carregados; o problema visual anterior foi resolvido (captura fornecida após a validação final).